Extraire l'identité du patient à grande échelle

NER & LLM fine-tuné

Alexandre Rogojan

Lifen · stage Machine Learning Engineer · partie 2

Le problème

Volume

Beaucoup d'ordonnances, de comptes rendus et de notes cliniques à traiter par mois [1].

Temps réel

Modèle doit être en mesure de répondre le plus rapidement possible [1].

Fiabilité & confidentialité

Erreur = dossier rangé chez un.e autre. [2].

		Principe
1	**NER**	encodeur
2	**LLM**	décodeur

1.1 NER : méthode, entrée et sortie

Méthode. Une étiquette à chaque mot, schéma BIO [3] : B = début d'une entité, I = suite, O = hors entité.

J'ai	revu	Monsieur	**Jean**	**Pierre**	**DUPONT**
O	O	O	**B-PRÉNOM**	**I-PRÉNOM**	**B-NOM**

Signé : Dr	**Irène**	**ADLER**
O	**B-PRÉNOM**	**B-NOM**

Entrée [4]

- Encodeur = on envoie en entrée une série de mots (un vecteur de mots) : Texte extrait du PDF.

Fonctionnement et sortie [4]

- Sortie : vecteur de labels, un label par mot.

1.2 NER : apprentissage

1. **Apprentissage supervisé :** dataset (= texte noté $x = (w_1, \ldots, w_n)$) d'entraînement annoté à la main : chaque mot w_i reçoit un label $y_i^\star$.

2. **Entraînement.** Descente de gradient, pour minimiser la cross-entropy :

$$\mathcal{L}(\theta) = -\sum_{i=1}^{n} \log p_\theta(y_i^\star \mid x, i)$$

   Le but pour le modèle est de donner une distribution de probabilité associée à chaque mot sur les labels.

1.3 NER : forces, limites et cas limites

Forces :

- Une passe, rapide, possible sur CPU.
- N'invente pas de nom : il met des labels sur les mots du texte.
- Score de rappel de 98,4 % sur les prénoms [AP-HP] [4].

Limites :

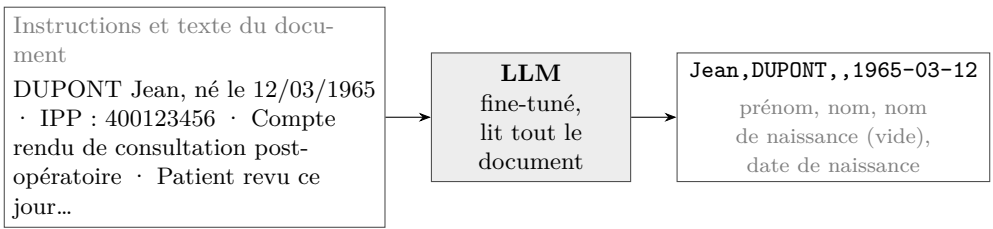
- Chaque mot du dataset d'entraînement doit être annoté à la main : 3 682 documents à AP-HP [4].
- Nécessité de classifier pour trouver le patient parmi les candidats
- Dépend de l'extraction du texte du PDF.

Cas :

- Nom de naissance : « DUPONT, née MARTIN ».
- Noms composés, initiales : LE GALL, J. DUPONT.

2.1 LLM :

- Le NER demande à des humains d'annoter chaque mot à la main. Demande aussi une normalisation robuste du texte (faire attention à la ponctuation, majuscules,...).



Avantages :
Apprentissage robuste (le LLM gère de nombreux cas de documents médicaux à la fois).

Inconvénients :
Demande un GPU, inférence lente, **hallucinations**, un serveur d'inférence.

2.2 LLM : comment structurer sa sortie ?

Solutions	**Principe**	**Limite**
Pré-prompt	on lui donne les instructions ”ta sortie doit être ...”	peu fiable, le modèle enfreint souvent les règles de son pré-prompt [1]
Décodage contraint	interdire les tokens hors format (outlines, guidance, instructor ; schémas JSON des fournisseurs)	ralentit beaucoup la phase d’inférence [5]
Fine-tuning	le modèle apprend le format à partir d’exemples	demande des données annotées

2.3 Temps d'inférence :

- **Le problème** : sans compression des poids, une prédiction prend près d'une seconde [1].
- **Cause** : au décodage, chaque nouveau token oblige le GPU à relire tous les poids. :

$$t_{\text{token}} \approx \frac{N \cdot b}{B}$$

N : nombre de paramètres
b : octets par paramètre
B : bande passante mémoire

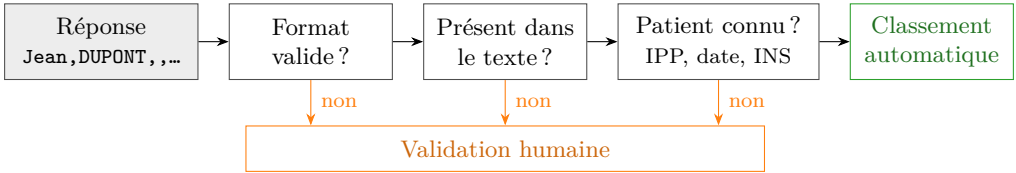
Format	Poids	t_{token}
BF16	6 Go	≈ 20 ms
FP8	3 Go	≈ 10 ms
INT4	1,5 Go	≈ 5 ms

- **Solution = Compression des poids** : permet l'écriture des poids avec avec arrondi, prenant moins de place dans la mémoire vidéo.
- **Tradeoff** : perte en précision.

2.4 Méthodes pour la compression : FP8 dynamique, AWQ, GPTQ

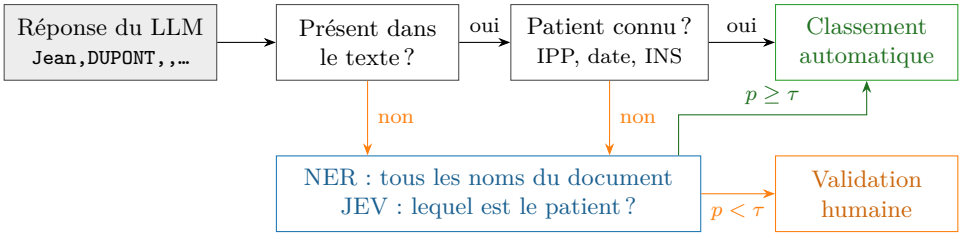
- **AWQ** (Activation-aware Weight Quantization) [6] : En pratique seuls $\approx 1\,\%$ des poids sont réellement significatifs et participent réellement à l'inférence (repérés par la taille relative des activations).
- **GPTQ** (Generalized Post-Training Quantization) [7] : poids = solutions d'un problème d'optimisation.
- **FP8 dynamique** : poids en FP8, compression adaptative à la nature du token en entrée.

2.5 Hallucinations LLM en sortie :



- **Noms inventés** : un décodeur peut écrire un nom absent du document. Ancrage : chaque nom n de la réponse doit être dans le document D, $\min_{w \in D} \mathrm{dist}(n, w) \leq k$. [5].
- **Besoin de vérification humaine** : dans [1] un vérificateur humain intervient dans les cas où le nom est halluciné.

2.6 Corriger une hallucination sans vérification humaine → JEV



- **Nom halluciné ou absent** : modèle NER pour trouver tous les noms du document, puis JEV choisit le patient parmi eux, avec une probabilité calibrée.
- **Cas de vérification humaine** : seulement si JEV est peu confiant ($p < \tau$). Piste récente : les modèles System One datent de 2026 [8].

2.7 TypeSafe JEV

Entrée : un état et des questions (JSON).

```
état = {
  "texte": "Madame Juliette MARTIN…",
  "candidats": ["Juliette MARTIN",
                "Nicolas JACQUES"]}
question = {"type": "choice",
  "instructions": "Qui est le patient
                   dans `texte` ?",
  "criteria": {
    "Juliette MARTIN": "…",
    "Nicolas JACQUES": "…"}}
```

Sortie : l'option choisie, une probabilité p_c par option, une confiance :

$$\hat{c} = \arg\max_c p_c, \quad \text{confiance} = \frac{p_{\max} - 1/n}{1 - 1/n}$$

- **System One** [8] : pas de texte en sortie, des probabilités sur des questions typées. La réponse est toujours une des options, mais peut être fausse.
- **Quatre types d'outputs** : `choice` (une option donnée), `noul` (probabilité du oui), `score` (un niveau), `box` (une zone d'image).
- **Calibré** : probabilités entraînées sur les issues réelles (RLCD [9]) ; vrai sur un groupe, pas pour chaque réponse [8].
- **Limites** : modèles open-source ?

3 Synthèse :

	NER (AP-HP)	**LLM & vérification JEV**
Mesure publiée	$F_1 = 98{,}6\,\%$ sur les prénoms [4]	?
Rappel publié	oui : 98,4 % (prénoms)	non
Courbe ROC	?	?
Sortie	labels sur les mots	les quatre champs, mis en forme
Code autour	choisir le patient, mettre en forme	contrôles de la sortie
Matériel	CPU possible	un GPU
Noms inventés	impossible	possibles : contrôles, JEV open-source ?

Tâches et données différentes : ces scores ne sont pas directement comparables entre colonnes.

Références

[1] G. Roy, E. Aouad. « Small LLMs, Big Gains: Simpler Patient Extraction with 85% Fewer Errors ». Lifen AI (Substack), 27 juillet 2026.

[2] Lifen. « Intelligence artificielle ». Page lifen.fr, consultée en 2026.

[3] L. A. Ramshaw, M. P. Marcus. « Text Chunking using Transformation-Based Learning ». Workshop on Very Large Corpora, 1995.

[4] X. Tannier *et al.* « Development and validation of a natural language processing algorithm to pseudonymize documents in the context of a clinical data warehouse ». arXiv:2303.13451, 2023.

[5] B. Vignal. « Improving LLM Reliability in Healthcare Data Extraction ». Blog technique de Lifen, septembre 2025.

[6] J. Lin *et al.* « AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration ». MLSys, 2024.

[7] E. Frantar *et al.* « GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers ». ICLR, 2023.

[8] TypeSafe AI. « Jev » et « System One models ». Documentation, docs.typesafe.ai, 2026.

[9] InfoQ. « TypeSafe AI Releases Jev: a Decision-Only Model That Returns Typed Probabilities Instead of Text ». Octobre 2026.